

Technology Choices, Design Decisions & Trade-offs

Workshop Registration Service — the reasoning behind the build, and how over-registration is prevented

Deliverable: the written document requested in the brief | **Companion:** *Local Setup Instructions* | **Measured on:** Node v22.17.0, PostgreSQL 16, 10 concurrent requests

The brief asks for one short document covering six subjects, so this is organised to be read in order of importance: **\$1** maps each problem in the brief to what was built, **\$4** is the capacity rule and the measurement behind it, and **\$2, 3, 5–7** give the reasoning, the assumptions, what was skipped and what was verified. Read **\$1** and **\$4** if you only have a minute.

1. What the brief asked for, and what was built

The client described four concrete pains. Each one is answered by something in the running application — not by a UI-only workaround, because the brief is explicit that anything outside the roles table "must be refused by the backend".

Pain in the brief	How it is solved	Where
"24 people turned up for a workshop with 20 seats. Two of us promised the last seat at the same time."	Capacity is enforced inside a database transaction that first takes a row lock on the workshop, so simultaneous requests are serialised and only one of them can take the final seat. A full workshop puts the next person on a waitlist instead of overbooking. Measured before/after in section 4.	<code>registrations.service.ts</code>
"When someone cancels, nobody frees up the seat or records who did it."	Cancelling frees the seat and immediately offers it to the oldest waitlisted attendee; the row itself is never deleted and keeps who cancelled it and when. A shared audit trail records every change with actor and timestamp.	<code>RegistrationsService.cancel</code> , <code>AuditLog</code>
"Three types of user, no public signup, permissions refused by the backend."	The first ADMIN is seeded; only an ADMIN can create accounts. Every route is protected by a JWT guard plus a role guard, so a forbidden action is a 403 from the API even if the UI never offered the button.	<code>RolesGuard</code> , <code>@Roles()</code>
"I just want to see which workshops this week still have seats."	The overview filters by date range, status and "seats available only", using seat counts the API computes and returns with each workshop.	<code>DashboardView</code> , <code>GET /workshops</code>
Extra credit: audit trail and waitlist	<code>AuditLog</code> rows for account creation, workshop create/edit, registration, cancellation and promotion, surfaced in an Activity view; plus a third registration status, <code>WAITLISTED</code> , promoted automatically when a seat is freed.	<code>ActivityView</code> , <code>promoteNextWaitlist</code>

Two fields were added to the spreadsheet the client shared, because the brief invited suggestions: a unique `code` per workshop (it is the natural identifier the staff already quote on the phone) and a `status` of `SCHEDULED` / `COMPLETED` / `CANCELLED`. Timestamps (`createdAt`, `updatedAt`, `cancelledAt`) were added for free by the ORM.

2. Technology choices and why

The brief left the stack open but asked for the reasoning. Each choice below was made against a deadline of a few hours, so "fast to build correctly" and "easy for a reviewer to run" counted as much as elegance.

Choice	Why	Trade-off accepted
NestJS 12 + TypeScript (backend)	The brief's permission table is a routing concern, and Nest expresses it declaratively: <code>@UseGuards(JwtAuthGuard, RolesGuard)</code> plus <code>@Roles('ADMIN')</code> on the handler. DTO classes with <code>class-validator</code> give input validation for free, and dependency injection keeps the capacity logic in one testable service. TypeScript catches the role/status typos that a plain JavaScript Express app would only reveal at runtime.	More ceremony than Express for a small API (modules, providers, decorators). Worth it here because correctness of access control and transactions is the graded part.

Prisma 6 (ORM + migrations)	One schema file is the source of truth for the models <i>and</i> the migration history, so table changes are reviewable in the repo. The generated client is fully typed, and <code>prisma.\$transaction(async tx => ...)</code> is what makes the multi-statement capacity check expressible without hand-written connection management.	An abstraction over SQL. When the capacity rule needed a real row lock, that meant dropping to <code>\$queryRaw</code> for one statement (documented in the code) instead of staying inside the typed API.
PostgreSQL (not SQLite)	The one rule the client cares about most is a concurrency rule, and it is only provable with row-level locking: <code>SELECT ... FOR UPDATE</code> . Postgres also gives real concurrent write behaviour to test against, and it is what the free hosting tier provides. SQLite would have been zero-setup, but it serialises writes anyway and cannot express this lock, so "it passed locally" would have proven nothing.	The reviewer must have Postgres available. Mitigated by the setup document (three ways to get one, including a single Docker command) and by the fact that the free hosted database is the same engine.
JWT + bcrypt (passport-jwt)	Matches the client's "no public signup" model with no session store: the API is stateless, so a second instance needs no shared session. The role travels in the token, which is what the role guard reads. bcrypt keeps password hashes salted and slow; the API never returns a hash.	No server-side revocation: a stolen token stays valid until it expires (1 day). Passwords are hashed, tokens are stored in <code>localStorage</code> , which is readable by any XSS on the page — acceptable for an internal tool on a trusted network, and the first thing to change for a public one.
Next.js 16 (App Router, static export) + plain CSS	Everything the UI does needs a signed-in user, so there is nothing to render on a server: <code>output: "export"</code> ships the app as static files, which is free to host anywhere and needs no runtime. Plain CSS with custom properties kept the design consistent without adding a styling toolchain.	No SSR/SEO and no Next image optimisation (irrelevant for a logged-in internal dashboard). Deep links rely on <code>trailingSlash</code> so each route resolves to a real <code>index.html</code> .
Vitest + oxlint	The unit suite mocks the Prisma transaction, so the capacity, waitlist and double-cancel rules are asserted in milliseconds, and <code>npm test</code> is the only command a reviewer needs to know.	Not the Jest ecosystem — negligible for a handful of suites (see section 7).
Netlify + Render + Neon (deployment)	All three have a usable free tier and map onto the architecture: static files on a CDN, a Node service for the API, managed Postgres for the data. The blueprint lives in <code>render.yaml</code> so the API environment is reproducible rather than clicked together.	Free instances sleep (Render ~15 minutes idle, Neon suspends computes), so the first request after a pause takes up to a minute. The UI shows a plain "Unable to reach the server" message instead of hanging, and the setup document explains warming the API before a demo.

3. Design decisions

3.1 Access control lives on the server

Two guards run on every protected route: the JWT guard proves *who* the caller is, the role guard decides *whether* they may. Roles are attached to the handler with a decorator, so the permission table from the brief is readable straight off the controllers. The frontend also hides what a role cannot use (the sidebar is filtered by role, and the admin-only menu simply is not rendered for a manager), but that is convenience, never the control — hiding a button protects nothing.

Action	ADMIN	MANAGER	STAFF	Enforcement
Create accounts / set roles	Yes	403	403	<code>@Roles('ADMIN')</code> on <code>POST /users</code>
List accounts	Yes	403	403	<code>@Roles('ADMIN')</code> on <code>GET /users</code>
Add / edit workshops	403	Yes	403	<code>@Roles('MANAGER')</code> on <code>POST</code> and <code>PATCH /workshops</code>
View workshops & history	403	Yes	Yes	<code>@Roles('MANAGER', 'STAFF')</code> on the read routes
Register / cancel attendees	403	Yes	Yes	<code>@Roles('MANAGER', 'STAFF')</code> on <code>/registrations</code>
View the audit trail	Yes	Yes	Yes	Same roles as "view history" — the bonus trail is a read of what those two roles already cause
Anything else	403			No route is reachable without a valid token: a missing or expired token is answered by the JWT guard with 401

An ADMIN legitimately gets a 403 on `GET /workshops`, which looks surprising for a second. It is the literal reading of the brief's table — an administrator looks after staff accounts, workshop management belongs to the manager — and giving the admin

everything "because admins usually can" would contradict the requirement that anything unmarked be refused by the backend. Since it is a deliberate decision rather than a hidden behaviour, the UI tells the admin which role to use instead of showing a broken page.

3.2 API shape and error semantics

Controllers stay thin (parse, delegate, return) and services own the invariants. Input never reaches a service without passing a DTO, and the global validation pipe rejects anything unexpected, so an unvalidated field cannot be smuggled into a write. Status codes are used for what they mean, which lets the frontend show the server's own message instead of inventing one:

Code	When	Example seen by the user
400	Validation failed, or an unknown field was sent	"role must be one of ADMIN, MANAGER, STAFF"
401	No token, expired token, wrong password	"Invalid credentials"
403	Authenticated, but the role may not do this	"You do not have permission to perform this action."
404	Unknown workshop or registration id	"Workshop not found"
409	A unique value is already taken	"Workshop code POT-101 is already in use"

Seat counts are computed by the API (one grouped query) and returned *with* each workshop, so the list, the filters and the detail view all agree on the same numbers. The UI never adds up registrations itself — duplicating that arithmetic in the client is how two screens end up disagreeing about a full workshop.

3.3 The data model keeps history by construction

- **A registration is never deleted.** Cancelling changes its `status` to `CANCELLED` and stamps `cancelledById` and `cancelledAt`; `createdById` and `createdAt` have been there since registration. The brief's requirement ("the record is never deleted, history always accessible") is therefore a property of the table, not a rule someone has to remember to follow.
- **Waitlist is a status, not a second table.** `ACTIVE` / `WAITLISTED` / `CANCELLED` on one row means a person entering the queue and later being promoted is a single row changing state, so the history view shows one continuous story. A separate waitlist table would have needed a copy step on every promotion and would have split the audit trail.
- **AuditLog for everything else** (account creation, workshop create/edit, registration, cancellation, promotion), each row carrying actor, action, entity, entity id, a JSON detail string and a timestamp. One flat table that any screen can render was preferred over per-entity history tables.
- **Statuses are strings validated with `@IsIn(...)`**, not database enums. The project began on SQLite, which has no native enum, and the API's job is to reject nonsense anyway; adding a fourth status later is then a one-line change rather than a migration that has to rewrite a type.
- **Deliberate omission:** `attendeeEmail` is not unique per workshop, so the same person can be registered twice (two attendees genuinely sharing a phone or an email is normal at a front desk). If the client wants a duplicate check, it is one unique index plus a 409 — a deliberate choice, not an oversight.

4. How over-registration is prevented

The client's own words: "a workshop can never hold more active registrations than its capacity", and "two of us promising the last seat at the same time is exactly what we are trying to stop". That makes this the one rule where "it worked when I clicked it" proves nothing, so it was tested the way it fails: ten staff hitting the same workshop at the same instant. The first version of the code **did** overbook. What follows is the measurement, the reason, and the fix.

4.1 Why a capacity check is harder than it looks

Deciding whether a seat is free is a read-then-write decision: count the active registrations, then insert (or update) a row. Between the read and the write, another request can do the same read and see the same stale answer. This is a classic time-of-check-to-time-of-use (TOCTOU) race, and no amount of testing with one user at a time exposes it.

The tempting fix is "wrap it in a transaction". That is not sufficient, and it is worth being precise about why: PostgreSQL runs statements at `READ COMMITTED` by default, where each statement sees a snapshot of committed data and a `COUNT()` takes no lock on the rows it reads. Two overlapping transactions both count zero active rows, both conclude a seat is free, and both insert `ACTIVE`. The transaction gives atomicity, not mutual exclusion.

Measured, before the fix. A probe created a workshop with `capacity = 1` and fired 10 registrations at the same instant (threads released together by a barrier), then counted the rows by status:

Requests in flight	Capacity	Rows ending <code>ACTIVE</code>	Verdict
10 (simultaneous)	1	7	FAIL — 6 attendees overbooked

That is the client's Saturday morning, reproduced in a few milliseconds: 7 staff each told an attendee they had the last seat.

4.2 The fix: take the row lock before counting

The decision is serialised by locking the workshop row for the duration of the transaction, before the count runs. Every path that can change a seat count takes this lock first, so the check and the write inside one transaction cannot be interleaved with another:

```
// registrations.service.ts – first statement inside this.prisma.$transaction(...)
await tx.$queryRaw`SELECT id FROM "Workshop" WHERE id = ${workshopId} FOR UPDATE`;

// ...only now is the count trustworthy for this workshop:
const activeCount = await tx.registration.count({ where: { workshopId, status: 'ACTIVE' } });
const status = activeCount >= workshop.capacity ? 'WAITLISTED' : 'ACTIVE';
await tx.registration.create({ data: { workshopId, attendeeName, attendeeEmail, status, createdById:
userId } });
```

What happens when 10 requests arrive together: the first transaction takes the lock and reads a count of 0, so it takes the seat; the other nine block on the same row. As each one is released it re-reads the count *after* the previous transaction committed — it now sees 1 of 1, so it writes `WAITLISTED` instead of a second `ACTIVE`. Nobody is turned away with an error and nobody is overbooked: the tenth person is queued, which is exactly the behaviour the waitlist bonus asks for. The lock is a single line, the id is parameterised (the tagged template becomes `$1`, never string interpolation), and no application-level mutex is involved — which matters, because an in-process lock would silently stop working the moment a second API instance existed.

Measured, after the fix (same probe, run three times):

Run	Requests in flight	Capacity	Rows ACTIVE	Rows WAITLISTED	Verdict
1	10	1	1	9	PASS
2	10	1	1	9	PASS
3	10	1	1	9	PASS

Identical outcome every run, which is the property that matters: the result no longer depends on timing.

4.3 Cancelling is the mirror image, and gets three guards

Freeing a seat has the same shape as taking one, plus a second risk: a seat that is freed twice can promote two people into it. Cancelling therefore does all of this inside one transaction:

1. **Lock first, then re-read.** The workshop row is locked before the registration's status is read, so the "was a seat actually freed?" decision cannot be made concurrently by two requests.
2. **Cancelling an already-cancelled registration is a no-op** that returns the existing row. It cannot free a second seat, and it cannot overwrite the original `cancelledById` / `cancelledAt` — the history keeps the person who really did it.
3. **Promotion happens only when an ACTIVE seat was actually freed**, and only while the count is still below capacity. Promoting the oldest waitlisted attendee (`ORDER BY createdAt ASC`) is the queue the front desk expects, and it records a `PROMOTE_WAITLIST` audit entry so the promotion is as traceable as the cancellation.

End-to-end on the seeded full workshop: 4 seats, 4 active, 1 waitlisted. Registering a fifth attendee returns `WAITLISTED`; cancelling one active registration leaves 4 active again with the waitlisted attendee promoted, the cancelled row still present with the actor and timestamp, and two new audit entries.

4.4 Alternatives considered, and why this one

Option	Would it work?	Why it was not chosen
SERIALIZABLE isolation + retry loop	Yes — Postgres aborts one of the conflicting transactions and the retry re-checks.	Every write path needs retry handling, and serialisation failures become a normal, user-visible code path. More code and less predictable latency than one lock on one row.
Unique partial index ("one ACTIVE row per seat")	No. An index can forbid duplicate <i>values</i> , but "at most capacity rows per workshop, and capacity changes" is not expressible.	Wrong tool: the rule is a count, not a uniqueness constraint.
Version column on Workshop + optimistic retry	Yes with a retry loop on conflict.	Same retry cost as SERIALIZABLE, plus a schema change and app-level conflict handling for a rule that needs no concurrency at all.
Database trigger / CHECK constraint counting rows	Possible with a trigger.	Hand-written SQL outside the Prisma migration flow, harder to test at the API level and to explain to the next developer. It is the natural belt-and-braces addition if other write paths ever appear.

In-process mutex, or a Redis / advisory lock	Only in-process would work, and only on a single instance.	A JavaScript mutex stops protecting anything the moment the API is scaled to two instances — a latent bug that looks fixed. Redis adds infrastructure to solve a problem Postgres already solves; an advisory lock is better, but it is the same Postgres feature with a less obvious name.
--	--	---

Honest limitation. The guarantee holds for every path that takes the lock, which today is both write paths (register and cancel). If a future endpoint inserts an `ACTIVE` registration without locking, the rule can be broken again — so the lock is documented in the service, and the DB-trigger option above remains the way to make it impossible rather than merely unimplemented.

4.5 Reproducing the measurement

The probe ships with the repository, so the claim above can be checked rather than trusted:

```
# API running and seeded (see the setup document), then:
python3 scripts/concurrency-check.py
# workshop <id> (RACE-...) capacity=1 -> 10 simultaneous registrations
# HTTP 201 ACTIVE: 1
# HTTP 201 WAITLISTED: 9
# DB rows: {'ACTIVE': 1, 'WAITLISTED': 9} | ACTIVE should be <= 1
# RESULT: PASS - capacity respected
```

To see the bug it prevents, remove the `SELECT ... FOR UPDATE` line from `backend/src/registrations/registrations.service.ts`, rebuild, and run the same command: the `ACTIVE` count jumps above 1 because the count-then-insert window is open again.

5. Trade-offs and assumptions

5.1 Assumptions

- **Attendees have no accounts.** The brief says staff type a name and an email, so there is no attendee login, no password reset and no self-service booking. A registration row therefore stores the name and email as entered rather than pointing at a user.
- **The front desk is trusted and identifiable.** Every registration and cancellation is stamped with the signed-in staff member, so accountability comes from the account rather than from extra fields on the form.
- **Capacity is a seat count, not a fixed room.** Editing capacity below the current number of active registrations is allowed and never un-registers anyone; it simply stops further `ACTIVE` registrations until seats fall below the new limit. Blocking the edit outright would have been more work for less flexibility.
- **Workshop codes are unique** and are what staff quote on the phone, so a duplicate is refused (409) rather than silently accepted.
- **Times are stored in UTC** and rendered in the browser's local zone — correct for a centre with three locations in one time zone, and harmless otherwise.
- **One shared database, small scale.** Around 15 staff and a few workshops a week: no sharding, no cache layer and no read replicas, because none of it would earn its keep at that size.

5.2 Problems found while building, and how each was solved

These surfaced during the build and are fixed in the committed code — listed because the diagnosis is the interesting part, not the diff.

Problem	Effect if left alone	Fix
Over-registration race — count-then-insert under <code>READ COMMITTED</code>	7 active rows on a 1-seat workshop: the exact failure the client hired us to stop.	Row lock (<code>SELECT ... FOR UPDATE</code>) taken before the count, in both write paths. Section 4.
Hardcoded JWT secret ('super-secret', in the signing and the verifying code)	The worst kind of bug: everything works, so nothing looks broken — yet anyone who can read the repository can sign themselves an <code>ADMIN</code> token and call every endpoint. It also ignored the secret the deployment generated.	One source of truth (<code>jwt.constants.ts</code>) reading <code>JWT_SECRET</code> from the environment, with a development-only fallback so local runs stay frictionless and production cannot fall back silently.

Validation gaps in the DTOs — role, status and capacity accepted any value; the update DTOs had no decorators at all	A typo such as capacity: 0 or an invented role reached the database, where it becomes a support ticket. The rules existed only in the interface.	Shared ROLES / WORKSHOP_STATUSES constants with @IsIn(...), plus @IsInt() @Min(1) on capacity and real decorators on the update DTOs. Bad input is now a 400 with a message.
Duplicate workshop code answered 500 ("Internal server error")	The manager filling in the form has no idea what went wrong, and it reads as a broken system rather than a rejected input.	Explicit uniqueness check on create, answered with a 409 naming the offending code.
The README described a different application (SQLite, a full workshop answered with an error, no waitlist, one seeded user)	A reviewer following the document would set up something that does not match the code — the fastest way to lose confidence in a submission.	Setup, seeding, roles and the capacity behaviour documented as they actually are, with the longer explanations moved into this pair of documents.
Missing NEXT_PUBLIC_API_URL on Netlify shipped a site that silently called localhost:3001	The deploy works on the developer's laptop and is broken for everyone else — the classic reason a live link fails an interview.	The build now refuses to finish when the variable is missing or still local, and prints which environment variables it did see.

6. What was deliberately left out

- **Automated concurrency test in CI.** The probe exists and passes, but it runs as a script rather than in `npm test`, because it needs a live server and database. Wiring it into CI with a service Postgres is the first thing I would add.
- **Notifying a promoted attendee** (email or SMS) and **real-time seat counts**. Both are visible on the next refresh, and the capacity rule is enforced in the database either way, so these are polish rather than correctness.
- **Pagination, export and log retention** for the activity view and registration history. Correct at demo scale, and the next step before the log covers years of entries.
- **Refresh tokens, revocation, login rate limiting**, and an accessibility audit. A single 1-day token and keyboard-reachable controls are proportionate for an internal tool; a public product would want the full set.

7. What was verified, and what each check proves

Check	Result	Proves
<code>npm test</code> (Vitest, backend)	6 tests passing (2 files)	Waitlist decision when a workshop is full; the row lock taken <i>before</i> the count and the insert; 404 for an unknown workshop with no row written; double-cancel as a no-op that frees nothing and overwrites nothing; promotion of the oldest waitlisted attendee.
<code>npm run build</code>	Compiles clean	The TypeScript in the submission type-checks end to end, so a reviewer's first command does not fail.
<code>npm run lint</code> (oxlint)	0 warnings, 0 errors (30 files)	No obvious mistakes or dead code left in <code>src/</code> and <code>test/</code> .
Concurrency probe × 3	10 simultaneous requests, capacity 1 → 1 ACTIVE / 9 WAITLISTED every run	The capacity rule survives simultaneous requests — the one thing the brief says it cares about most.
Permission matrix over HTTP	403 for every role/action the brief marks "No"; 401 without a token	Access control is enforced by the API, not by the interface.
Waitlist round trip on the seeded full workshop	Register → WAITLISTED; cancel → promoted; cancelled row kept with actor and time	History and the waitlist bonus work together on real data, with no manual database edits.

The reasoning in one sentence: the stack was chosen so the two graded rules — who may do what, and never selling the same seat twice — are each enforced in exactly one place (a role guard per route, a row lock per workshop), with everything else kept as plain as possible so the project runs the first time somebody else tries it.